



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251235
Nama Lengkap	NATALIA GRECIA PUTRI
Minggu ke / Materi	09 / Manajemen File

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

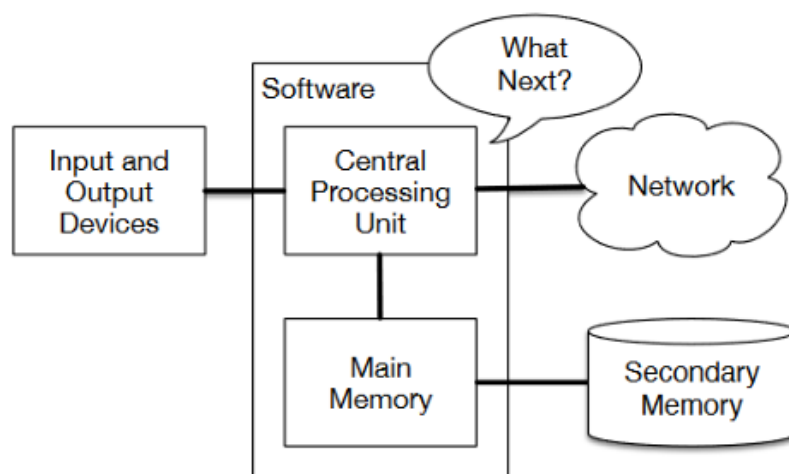
BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Pengantar File

Setiap kali sebuah program dijalankan di komputer, program tersebut memerlukan memori utama (RAM) sebagai tempat kerja sementara. Seluruh data dan instruksi yang dibutuhkan program akan dimuat dan diproses di dalam RAM selama program berjalan. Namun, RAM memiliki satu kelemahan mendasar, yaitu bersifat volatile atau sementara, artinya semua data yang tersimpan di dalamnya akan hilang begitu komputer dimatikan atau mengalami gangguan daya.

Untuk mengatasi masalah ini, dibutuhkan memori sekunder sebagai tempat penyimpanan data yang bersifat permanen.



Gambar 10.1: Secondary Memory di Komputer

Diagram pada gambar menunjukkan alur antara:

- Input/Output Devices (keyboard, mouse, monitor, dll)
- CPU (Central Processing Unit): Otaknya komputer
- Main Memory (RAM): Tempat menyimpan data sementara
- Secondary Memory: Penyimpanan permanen
- Network: Koneksi ke jaringan/internet

- Software: Program yang berjalan

File adalah kumpulan data yang disimpan di memori sekunder. File bisa berupa:

- Teks (misal: .txt)
- Gambar (misal: .jpg, .png)
- Multimedia (misal: .mp3, .mp4)
- Binary, file sistem, atau file program

File juga punya **atribut properti**, seperti:

- Nama file
- Ukuran file
- Lokasi penyimpanan
- Hak akses
- Tanggal dibuat dan diubah

Contoh Code Menyimpan dan Membaca File

```
with open("contoh.txt", "w") as file:
    file.write("Halo, ini adalah data yang disimpan di file.\n")
    file.write("Data ini akan tetap ada meskipun program dimatikan.")

with open("contoh.txt", "r") as file:
    isi = file.read()
    print("Isi file:")
    print(isi)
```

Penjelasan:

open("contoh.txt", "w"): Membuka atau membuat file contoh.txt untuk ditulis.

write(): Menulis teks ke dalam file.

open("contoh.txt", "r"): Membuka file untuk dibaca.

read(): Membaca seluruh isi file.

Pengaksesan File

Agar Python dapat berinteraksi dengan file (baik membaca isinya maupun menulis data ke dalamnya) terdapat empat langkah dasar yang harus dilakukan secara berurutan:

1. Siapkan file dan path-nya, yaitu menentukan lokasi atau alamat file di dalam sistem komputer.

2. Buka file menggunakan file fungsi `open()` dengan menentukan namafile danmode aksesnya.
3. Lakukan operasi yang diinginkan pada file, seperti membaca dengan `read()`, menulis dengan `write()`, dan sebagainya.
4. Tutup file setelah selesai menggunakan `close()` agar tidak terjadi kebocoran memori.

Contoh:

Membaca file untuk dibaca

```
file_handle = open('mbox-short.txt', 'r')
print(file_handle)
```

Output

```
<_io.TextIOWrapper name='mbox-short.txt' mode='r' encoding='cp1252'>
```

Membaca isi file

Menggunakan `with` statement sangat dianjurkan karena Python akan otomatis menutup file begitu blok kode di dalamnya selesai dieksekusi, tanpa perlu memanggil `close()` secara manual. Hal ini penting untuk mencegah memory leak dan berbagai masalah akses file yang dapat terjadi jika file diberikan tetap terbuka.

```
with open('mbox-short.txt', 'r') as file:
    for line in file:
        print(line.strip())
try:
    file_handle = open('tidak-ada.txt', 'r')
except FileNotFoundError:
    print("File tidak ditemukan!")
```

Output:

```
From: stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
Date: Sat, 5 Jan 2008 09:12:18 -0500
To: source@collab.sakaiproject.org
From: stephen.marquard@uct.ac.za
Subject: [sakai] svn commit: r39772 - content/branches/
Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772
```

Jika file tidak ditemukan, Python akan menghasilkan error:

```
try:
    file_handle = open('tidak-ada.txt', 'r')
except FileNotFoundError:
    print("File tidak ditemukan!")
```

Output:

```
Traceback (most recent call last):
  File "c:\Users\ACER\OneDrive\Documents\file_marfi\SEM2\PrAlPro\LAPORAN9\TM9\test.py", line 1, in <module>
    with open('mbox-short.txt', 'r') as file:
        ~~~~~^
FileNotFoundError: [Errno 2] No such file or directory: 'mbox-short.txt'
```

Manipulasi File

Membuka dan Menutup File

Sebelum melakukan operasi apapun pada sebuah file, langkah pertama yang harus dilakukan adalah membukanya menggunakan fungsi `open()`. Setelah semua operasi selesai, file wajib ditutup menggunakan `close()` untuk membebaskan sumber daya yang digunakan.

Contoh:

```
file = open('mbox-short.txt', 'r')
print(file)
file.close()
```

Output:

```
<_io.TextIOWrapper name='mbox-short.txt' mode='r' encoding='cp1252'>
PS C:\Users\ACER\OneDrive\Documents\file_marfi\SEM2\PrAlPro\LAPORAN9\TM9>
```

Menghitung Jumlah Baris dalam File

Kita bisa menggunakan loop untuk menghitung total baris dalam file.

```
with open('mbox-short.txt', 'r') as file:
    line_count = 0
    for line in file:
        line_count += 1
    print("Total baris:", line_count)
```

File txt:

```

1 From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
2 Return-Path: <postmaster@collab.sakaiproject.org>
3 Date: Sat, 5 Jan 2008 09:12:18 -0500
4 To: source@collab.sakaiproject.org
5 From: stephen.marquard@uct.ac.za
6 Subject: [sakai] svn commit: r39772 - content/branches/
7 Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772

```

Output:

```

0(LAPORAN\TMS\test.p
Total baris: 7

```

Memfilter Baris Tertentu (Contoh: Baris yang Dimulai dengan "Date:")

Kita bisa menggunakan `str.startswith()` untuk memfilter baris tertentu.

Menampilkan 10 Baris Pertama yang Dimulai dengan Date:

```

with open('mbox-short.txt', 'r') as file:
    count = 0
    for line in file:
        if line.startswith("Date:") and count < 10:
            print(line.rstrip())
        count += 1

```

Output:

```

Date: Sat, 5 Jan 2008 09:12:18 -0500
Date: 2008-01-05 09:12:07 -0500 (Sat, 05 Jan 2008)
Date: Fri, 4 Jan 2008 18:08:57 -0500
Date: 2008-01-04 18:08:50 -0500 (Fri, 04 Jan 2008)
Date: Fri, 4 Jan 2008 16:09:02 -0500
Date: 2008-01-04 16:09:01 -0500 (Fri, 04 Jan 2008)
Date: Fri, 4 Jan 2008 15:44:40 -0500
Date: 2008-01-04 15:44:39 -0500 (Fri, 04 Jan 2008)
Date: Fri, 4 Jan 2008 15:01:38 -0500
Date: 2008-01-04 15:01:37 -0500 (Fri, 04 Jan 2008)

```

- `.rstrip()` digunakan untuk menghilangkan spasi/newline di akhir baris.
- Tanpa `.rstrip()`, akan muncul double newline karena:
- File asli sudah memiliki `\n` di akhir setiap baris.
- Fungsi `print()` juga menambahkan `\n` secara default.

Mengambil Karakter Terakhir dari File

Kita bisa menggunakan slicing untuk mengambil bagian tertentu dari isi file.

```
with open('mbox-short.txt', 'r') as file:
    content = file.read()
    last_16_chars = content[-16:]
    print("16 karakter terakhir:", last_16_chars)
```

Output:

```
16 karakter terakhir: > Preferences.
```

Penyimpanan File

Untuk menyimpan atau menuliskan data ke dalam sebuah file, Python menggunakan fungsi `open()` dengan mode `'w'` (write). Mode ini berbeda dengan mode `'r'` (read) yang hanya digunakan untuk membaca, mode `'w'` memungkinkan kita menuliskan data baru ke dalam file.

Contoh:

```
handle = open('output.txt', 'w')
tulisan = "teks ini akan dituliskan ke file\n"
handle.write(tulisan)
handle.close()
```

Penjelasan:

- `open('output.txt', 'w')`

Membuka file `output.txt` dalam mode write (`'w'`)

Jika file sudah ada, isinya akan dihapus/ditimpa

Jika file tidak ada, akan dibuat baru

- Variabel `tulisan`

Menyimpan string yang akan ditulis ke file

`\n` adalah karakter newline (baris baru)

- `handle.write(tulisan)`

Menulis isi variabel `tulisan` ke file

Tidak otomatis menambahkan newline, harus ditulis manual (`\n`)

- `handle.close()`

Menutup file setelah selesai

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

https://github.com/grec12/71251235_nataliagrecia.git

```
1  file1 = open('file1.txt', 'r', encoding='utf-8')
2  file2 = open('file2.txt', 'r', encoding='utf-8')
3
4  line1 = file1.readlines()
5  line2 = file2.readlines()
6
7  file1.close
8  file2.close
9
10 jumlah_baris = max(len(line1), len(line2))
11
12 for i in range(jumlah_baris):
13     teks1 = line1[i].strip() if i < len(line1) else ''
14     teks2 = line2[i].strip() if i < len(line2) else ''
15
16     if teks1 == teks2:
17         print(f"Baris {i+1} SAMA: {teks1}")
18     else:
19         print(f"Baris {i+1} BERBEDA: ")
20         print(" file1:", teks1)
21         print(" file2:", teks2)
22     print()
```

```
Baris 1 BERBEDA:
file1: Soobin
file2: Choi Soobin

Baris 2 BERBEDA:
file1: Yeonjun
file2: Choi Yeonjun

Baris 3 BERBEDA:
file1: Beomgyu
file2: Choi Beomgyu

Baris 4 BERBEDA:
file1: Taehyun
file2: Kang Taehyun

Baris 5 SAMA: Hueninkai

Baris 6 SAMA: Tomorrow By Together
```

Penjelasan kode:

Membaca file:

- file1 dan file2 dibuka dalam mode baca ('r') dengan encoding UTF-8
- Seluruh baris dari masing-masing file dimuat ke dalam variabel line1 dan line2 menggunakan readlines().

Menutup file:

- Setelah data selesai dibaca, kedua file ditutup untuk mencegah pemborosan sumber daya sistem.

Menentukan jumlah iterasi:

- Jumlah_baris diisi dengan nilai terbesar antara baris file1 dan file2 menggunakan max(), sehingga seluruh baris dari kedua file tetap dibandingkan.

Perulangan for i in range(jumlah_baris):

- Mengambil baris: baris ke-i diambil dari line1 dan line2. Jika salah satu sudah kehabisan baris, posisinya diisi string kosong "".
- Menghilangkan spasi: .strip() digunakan agar spasi diawal dan akhir baris tidak memengaruhi hasil perbandingan.
- Membandingkan:
 - Jika isi kedua baris sama, ditemukan: Baris x SAMA: [teks].
 - Jika berbeda, ditampilkan: Baris x BERBEDA: beserta isi masing-masing baris dari kedua file.
 - Baris kosong dicetak setelah setiap hasil perbandingan sebagai pemisah.

SOAL 2

```
1 file_name = 'soal.txt'
2 print("nama file1:", file_name)
3
4 with open(file_name, 'r', encoding= 'utf-8') as file:
5     for line in file:
6         if '||' in line:
7             soal, kunci = line.strip().split('||')
8             soal = soal.strip()
9             kunci = kunci.strip().lower()
10            print(soal)
11            answer = input("Jawab: ").strip().lower()
12            if answer == kunci:
13                print("Jawaban benar!\n")
14            else:
15                print("Jawaban salah!\n")
```

```
nama file1: soal.txt
1+1 =
Jawab: 2
Jawaban benar!

Bendera Indonesia?
Jawab: merah putih
Jawaban benar!

Kota gudeg adalah:
Jawab: jogja
Jawaban salah!

Komponen PC untuk penyimpanan file adalah...
Jawab: harddisk
Jawaban benar!

50 * 20 =
Jawab: 1000
Jawaban benar!
```

Penjelasan:

- Program di atas membaca soal dari soal.txt.
- Memisahkan pertanyaan dan jawaban menggunakan ||.
- Baris diproses satu persatu menggunakan perulangan.
- Menerima dan mengevaluasi jawaban:
 - Soal ditampilkan ke layar, lalu pengguna diminta mengetikkan jawaban.
 - Jawaban yang masuk juga dibersihkan dari spasi dan diubah ke huruf kecil sebelum dibandingkan dengan kunci.
 - Jika jawaban cocok dengan kunci, program mencetak "Jawaban benar!".
 - Jika tidak cocok, program mencetak "Jawaban salah!".
 - Proses ini berulang hingga seluruh baris dalam file selesai diproses.